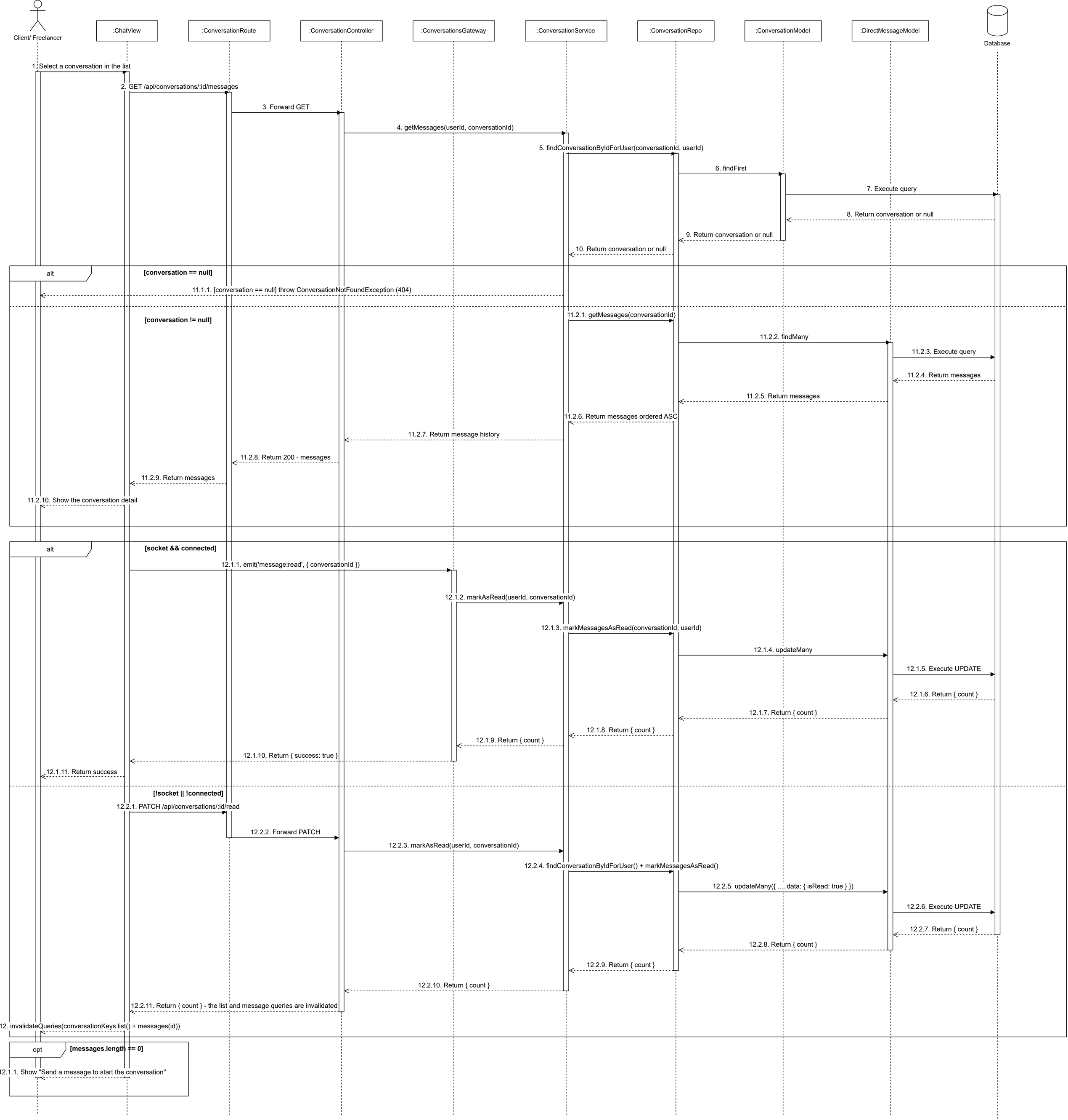

Client/ Freelancer
:ChatView
:ConversationRoute
:ConversationController
:ConversationsGateway
:ConversationService
:ConversationRepo
:ConversationModel
:DirectMessageModel
Database
1. Select a conversation in the list
2. GET /api/conversations/:id/messages
3. Forward GET
4. getMessages(userId, conversationId)
5. findConversationByIdForUser(conversationId, userId)
6. findFirst
7. Execute query
8. Return conversation or null
9. Return conversation or null
10. Return conversation or null
alt
[conversation == null]
11.1.1. [conversation == null] throw ConversationNotFoundException (404)
[conversation != null]
11.2.1. getMessages(conversationId)
11.2.2. findMany
11.2.3. Execute query
11.2.4. Return messages
11.2.5. Return messages
11.2.6. Return messages ordered ASC
11.2.7. Return message history
11.2.8. Return 200 - messages
11.2.9. Return messages
11.2.10. Show the conversation detail
alt
[socket && connected]
12.1.1. emit('message:read', { conversationId })
12.1.2. markAsRead(userId, conversationId)
12.1.3. markMessagesAsRead(conversationId, userId)
12.1.4. updateMany
12.1.5. Execute UPDATE
12.1.6. Return { count }
12.1.7. Return { count }
12.1.8. Return { count }
12.1.9. Return { count }
12.1.10. Return { success: true }
12.1.11. Return success
[!socket || !connected]
12.2.1. PATCH /api/conversations/:id/read
12.2.2. Forward PATCH
12.2.3. markAsRead(userId, conversationId)
12.2.4. findConversationByIdForUser() + markMessagesAsRead()
12.2.5. updateMany({ ..., data: { isRead: true } })
12.2.6. Execute UPDATE
12.2.7. Return { count }
12.2.8. Return { count }
12.2.9. Return { count }
12.2.10. Return { count }
12.2.11. Return { count } - the list and message queries are invalidated
12.2.12. invalidateQueries(conversationKeys.list() + messages(id))
opt
[messages.length == 0]
12.1.1. Show "Send a message to start the conversation"